Ders 12 Genişletilmiş Notu: Seminer Sunumları: DeepZip, FastText.zip, Fraktal Görüntü Sıkıştırma ve RNN ile Görüntü Sıkıştırma

Ders 12 Genişletilmiş Notu: Seminer Sunumları: DeepZip, FastText.zip, Fraktal Görüntü Sıkıştırma ve RNN ile Görüntü Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: DeepZip, Tekrarlayan Sinir Ağlarıyla Kayıpsız Sıkıştırma
- 3. Sunum 2: FastText.zip, Metin Sınıflandırma Modelinin Sıkıştırılması
- 4. Sunum 3: Fraktal Görüntü Sıkıştırmaya Giriş (İngilizce sunum)
- 5. Sunum 4: Fraktal Görüntü Sıkıştırma (aynı makalenin Türkçe sunumu)
- 6. Sunum 5: Tekrarlayan Sinir Ağlarıyla Tam Çözünürlüklü Görüntü Sıkıştırma
- Örnek Sorular

1. Dersin Yapısı

Bu ders, bir önceki dersle başlayan seminer haftalarının ikinci dersidir. Her sunumda bir öğrenci, veri sıkıştırmayla ilgili yayımlanmış bir makaleyi ekran paylaşarak anlattı; sunum süresi yaklaşık 20 dakikaydı. Hoca sunumlar sırasında anlamadığı yerleri sordu, sunumdan sonra da konuyu derste görülen kavramlara bağlayan sorular yöneltti.

Bu derste beş sunum yapıldı. İlk ikisi sinir ağlarının ve nicemlemenin sıkıştırmada kullanılmasıyla ilgiliydi: RNN ile kayıpsız sıkıştırma (DeepZip) ve bir metin sınıflandırma modelinin küçültülmesi (FastText.zip). Üçüncü ve dördüncü sunumda iki öğrenci, sonradan aynı makale olduğu anlaşılan bir fraktal görüntü sıkıştırma çalışmasını anlattı. Son sunum, Google Research'ün RNN'lerle tam çözünürlüklü görüntü sıkıştırma çalışmasıydı.

Üçüncü sunum İngilizce yapıldı ve altyazının bu bölümü (yaklaşık 00:41:00-00:51:30) çoğu yerde anlamsız tekrarlara düştü. Bu sunumun içeriği slaytlardan kuruldu; aynı makale dördüncü sunumda Türkçe ve ayrıntılı anlatıldığı için asıl anlatım 5. bölümdedir.

2. Sunum 1: DeepZip, Tekrarlayan Sinir Ağlarıyla Kayıpsız Sıkıştırma

İlk sunumda bir öğrenci, "DeepZip" adlı, tekrarlayan yapay sinir ağlarıyla (RNN) kayıpsız sıkıştırma yapan makaleyi anlattı. Hoca girişte konunun çok güncel olduğunu belirtti.

> **Ek bilgi (derste anlatılmadı):** Makale büyük olasılıkla Goyal ve arkadaşlarının 2019'da Data Compression Conference'ta yayımlanan "DeepZip: Lossless Data Compression Using Recurrent Neural Networks" çalışmasıdır.

2.1 Temel fikir: iyi tahmin, iyi sıkıştırma

Makalenin çıkış noktası, iyi tahmin modellerinin iyi sıkıştırma da gerçekleştirdiğidir. Bir model verinin tümünü daha küçük temsillerle ifade ettiği için yaptığı iş aslında sıkıştırmaya benzer; bu yüzden tahmin modelleri bazı değişikliklerle veri sıkıştırmada kullanılabilir. RNN'ler uzun vadeli ilişkileri tespit etmekte ve gelecek sembolü tahmin etmekte başarılıdır. Çalışmada RNN tabanlı bir olasılık tahmin bloğu ile bir aritmetik kodlama bloğu birlikte kullanıldı; deneyler sentetik ve gerçek veri setlerinde yapıldı.

2.2 Entropi, aritmetik kodlama ve k-gram sınırı

Sunan öğrenci, büyük verinin gelişmesiyle sıkıştırmanın öneminin yeniden arttığını belirterek dersten bilinen kavramları hatırlattı. Shannon'a göre entropi, sembol başına gereken en az bit sayısını belirler. Aritmetik kodlama, verinin bütününe tek bir değer atayarak entropiye çok yakın bir sıkıştırma yapar; olasılık dağılımı bilindiğinde bunu hızla gerçekleştirir. Kod çözücü de aynı dağılımı bildiği için veriyi geri açabilir.

Dağılım bilinmediğinde de aritmetik kodlama kullanılabilir. Öğrenci burada derste görülen PPM'i örnek verdi: tablo dinamik olarak oluşur, sürekli güncellenir ve tablodan alınan olasılıklar aritmetik kodlayıcıya gönderilir. Bu yöntemlerde k-gramlar kullanılır. Ancak k arttıkça hesaplama maliyeti üstel olarak artar; bu yüzden k'ye bir sınır konur ve bu sınır uzun vadeli ilişkilerin yakalanmasını engeller. RNN'in önemi tam bu noktada ortaya çıkar: uzun vadeli ilişkileri yakalayabilir ve sonraki sembolü tahmin edebilir.

Literatürde RNN ile sıkıştırma yapan çalışmalar olumlu sonuçlar vermişti, ancak yalnız klasik RNN kullanıldığı için karmaşık verilerde düşük sıkıştırma oranları elde edilmişti. Öğrencinin andığı bir başka çalışmada RNN, birçok kodlayıcının çıktısını birleştiren bir bağlam karıştırıcısı (context mixer) olarak kullanılmıştı.

2.3 Mimari: RNN olasılık tahmin bloğu ve aritmetik kodlama bloğu

Önerilen kodlayıcı iki bloktan oluşur (ekrandan, 00:10:00):

- **RNN olasılık tahmin bloğu:** kendinden önceki bütün elemanlara ait gizli bir bilgi tutar ve sonraki elemanın olasılık dağılımını döndürür. Softmax katmanı bulunan herhangi bir RNN yapısı kullanılabilir: Vanilla RNN, LSTM ya da GRU.
- **Aritmetik kodlama bloğu:** o anki sembole göre aralığı değiştirir ve yeni bir duruma geçer.

İki blok da iteratif olarak güncellenir ve bilgilerini korur. LSTM ve GRU, iç tasarımı daha gelişmiş yapılardır; verinin bütününe yayılmış karmaşık ilişkileri daha iyi yakaladıkları için daha iyi sıkıştırma sağlarlar. Slayttaki çizimde üç hücre gösterildi: yalnız tanh içeren RNN hücresi; unutma (forget), giriş (input) ve çıkış (output) kapılı LSTM hücresi; sıfırlama (reset) ve güncelleme (update) kapılı GRU hücresi (ekrandan, 00:10:00).

Bu mimaride en önemli koşul, **kodlayıcı ile kod çözücünün aynı sembol için aynı dağılımı üretmesidir**. Kod çözücü kodlanan veriyi aynı şekilde açmak zorunda olduğundan, ağırlıklar iki tarafta da aynı biçimde güncellenir (ekrandan, 00:10:00).

Çalışmada iki model kullanıldı:

- **DeepZip-ChGRU:** RNN bloğu olarak GRU kullanan model.
- **DeepZip-Feat:** girdi olarak yalnız sonraki elemanı vermek yerine, son belirli sayıda elemandan (örneğin son 20 eleman) ve bu elemanlar içindeki bazı bağlamların oluşma sayılarından bir özellik çıkarır (feature extraction) ve bu özellikle güncellenir. Daha karmaşık bir modeldir.

2.4 Aritmetik kodlamanın sonlu durum makinesi olarak görülmesi

Aritmetik kodlama bütün veriye 0 ile 1 arasında bir aralık döndürür; her farklı eleman dizisi için farklı bir sonuç çıkar ve bu tek sayıdan bütün dizi geri elde edilebilir. Makale aritmetik kodlamayı bir **sonlu durum makinesine** benzetir: kodlayıcının mevcut aralığı makinenin bulunduğu durumdur, gelen her eleman bu durumu değiştirir. Kodlamanın sonunda tek bir

değer sıkıştırılmış veriyi oluşturur; kod çözme bunun tersidir. Gelen sembollere göre aralık sürekli daralır.

2.5 Kodlama ve kod çözme akışı

İlk sembolde RNN henüz hiçbir sembol görmediği için bir şey öğrenmemiştir ve bir tahmin veremez. Bu yüzden aritmetik kodlayıcı ilk sembolde **varsayılan bir olasılık dağılımı** (default distribution) kullanır. Sunan öğrenci bunu PPM'de bilinmeyen sembol için $1/|\text{alfabe}|$ kullanılmasına benzetti.

Slayttaki akış şöyledir (ekrandan, 00:13:20):

```
Kodlayıcı:
  S0 -> Encoder (varsayılan dağılım)
  S0 -> RNN -> dağılım -> S1 -> Encoder
  S1 -> RNN -> dağılım -> S2 -> Encoder
  S2 -> RNN -> dağılım -> S3 -> Encoder -> Sıkıştırılmış veri

Kod çözücü:
  Sıkıştırılmış veri -> Decoder (varsayılan dağılım) -> S0
  S0 -> RNN -> dağılım -> Decoder -> S1
  S1 -> RNN -> dağılım -> Decoder -> S2
  S2 -> RNN -> dağılım -> Decoder -> S3
```

Kodlama sırasında her adımda hem RNN değişir ve bilgisini saklar, hem de aritmetik kodlayıcının aralığı güncellenir. Kod çözme de varsayılan dağılımla başlar, çünkü ilk sembol bu dağılımla kodlanmıştır. İlk eleman çözüldükten sonra S_0 kod çözücünün RNN'ine verilir; böylece RNN kodlayıcıdaki ile aynı sonucu üretir ve sonraki sembol çözülür.

2.6 Veri setleri

Deneyler sentetik ve gerçek veri setlerinde yapıldı (ekrandan, 00:14:45):

- **Sentetik veri setleri**
  - i.i.d. kaynaklar: bağımsız ve özdeş dağılmış rastgele değişkenler. Kolay bir veri setidir; çoğu sıkıştırıcı kısa sürede entropiye ulaştığı için makale bu sette karşılaştırma yapmamıştır.
  - Markov-k kaynakları: $X(n) = X(n-1) + X(n-k) \bmod M$ formülüyle üretilir. k arttıkça her eleman daha uzaktaki bir elemanla ilişkili olur; veri daha karmaşık ve sıkıştırılması daha zor hale gelir.
- **Gerçek veri setleri**
  - PSRN-m-n: sıkıştırılması zor sözde rastgele sayılar.
  - Hutter Prize: metin veri seti.
  - Enwiki9: büyük XML veri seti.
  - HGP-Chr1: insan genomu veri seti.

Sunan öğrenci, sıkıştırmanın kolay olmasının verinin öğrenilmesinin kolay olmasıyla ilişkili olduğunu vurguladı. İyi sınıflandırıcılar ve iyi tahmin mekanizmalarının sıkıştırmada da başarılı olmasının nedeni, verideki örüntüleri ve ilişkileri yakalamalarıdır.

2.7 Sonuçlar

Önce klasik RNN ile GRU karşılaştırıldı. Klasik RNN sıkıştırmaya başlar, ancak karmaşık verilerde **kaybolan gradyan** problemiyle erken karşılaşır; altyazıya göre Markov-35 verisinde artık sıkıştırma yapamaz, çünkü yapısı gereği uzun vadeli ilişkileri kaybeder. GRU daha karmaşık verilerde de sıkıştırma yapar ve bu problemle ancak Markov-60 civarında karşılaşır. DeepZip-Feat ise bu sınıra ancak Markov-140'ta ulaşır.

Slayttaki tablo, 100 MB'lık Markov kaynaklarının sıkıştırılmış boyutlarını DeepZip-128-Cell modelleri, GZIP ve uyarlamalı aritmetik kodlama (CABAC) için vermektedir (ekrandan, 00:17:05):

Veri seti	Boyut	DeepZip-ChGRU	DeepZip-Feat	GZIP	Adaptive-AE
Markov-20	100 MB	200 KB	200 KB	5 MB	2 MB
Markov-30	100 MB	1 MB	400 KB	40 MB	87 MB
Markov-50	100 MB	30 MB	500 KB	63 MB	100 MB
Markov-80	100 MB	100 MB	2 MB	69 MB	100 MB
Markov-100	100 MB	100 MB	38 MB	100 MB	100 MB
Markov-140	100 MB	100 MB	100 MB	100 MB	100 MB

Tablodaki 100 MB değeri, o yöntemin veriyi hiç sıkıştıramadığını gösterir. Genel olarak DeepZip modelleri neredeyse bütün veri setlerinde diğer sıkıştırıcılardan başarılıdır; GZIP karmaşık verilerde sıkıştırma yapamazken DeepZip-Feat bunlarda da sıkıştırma yapmıştır.

Gerçek veri setlerinde GZIP'in rastgele sayılarda başarısız olduğu, DeepZip'in ise iyi sonuç verdiği gösterildi. Makalenin en çok vurguladığı nokta zor veri setlerindeki başarıdır: genom veri setinde DeepZip, literatürdeki en iyi sıkıştırıcıdan daha iyi sonuç verdi. Makale bunu verinin bütününe yayılmış karmaşık bağlantıları keşfetme gücüne bağlamıştır. Metin veri setlerinde ise en iyi sıkıştırıcı geçilemedi; makale bunu metinde çok uzun ve karmaşık bağımlılıkların olmamasıyla yorumladı.

Sunan öğrenci sonuç olarak RNN bloklarının derste görülen k-gramlara çok benzer biçimde kullanıldığını belirtti: RNN önceki bütün elemanların gizli bilgisini tutup bir sonraki sembol için bir olasılık verir, sembol geldiğinde onun olasılığı RNN'in tahmininden alınıp aritmetik kodlayıcıya gönderilir.

2.8 Soru-cevap

Soru: Makale çalışma süresi hakkında bilgi vermiş mi?

Cevap: Hayır. Makale literatürdeki bir çalışmayı klasik RNN kullandığı ve eğitim süresinin uzun olduğu için eleştirmiş, ancak kendi çalışma sürelerini vermemiştir. Sunan öğrenci bunun makalenin eleştirilebilecek bir yanı olduğunu söyledi.

Soru: "Zor veri seti" ile ne kastediliyor? Hiç tekrarı olmayan veri setleri mi?

Cevap: Örneğin Markov-k verisinde k arttıkça örüntü daha uzaktaki elemanlarla ilişkili hale gelir. Zor veri seti, içindeki ilişkilerin ve örüntülerin tespit edilmesi zor olan veridir. RNN bir şey öğrenemediğinde "şu ana kadar bu semboller geldi, bundan sonra şunun gelme olasılığı yüksek" diyemez.

Hoca bunu şöyle özetledi: bu yöntemde olasılıklar metin içinde ilerledikçe RNN modelinin içinde değişir; önceden hazırlanmış, çıkarılmış olasılık değerleri yoktur.

3. Sunum 2: FastText.zip, Metin Sınıflandırma Modelinin Sıkıştırılması

İkinci sunumda bir öğrenci, Facebook ekibinin FastText metin sınıflandırma modelini sıkıştırdığı "FastText.zip" çalışmasını anlattı. Sunuma göre makale 2016 tarihlidir.

3.1 FastText ve amaç

FastText, Facebook tarafından geliştirilen bir metin sınıflandırma kütüphanesidir. Amacı standart donanımlarda da çalışabilmektir; bu yüzden iyi bir sıkıştırma ve performans önemlidir. Sunan öğrenci örnek olarak bir menüdeki yemekleri kahvaltı, öğle ve akşam yemeği diye sınıflayan bir modeli verdi: model daha sonra "tost ve reçel" gibi yeni bir girdinin hangi sınıfa ait olduğunu söyler. Haber, film yorumu gibi pek çok metin bu şekilde sınıflanabilir.

FastText.zip'in amacı, modelin doğruluğunu kaybetmeden bellek ve saklama maliyetini düşürmektir. Sunan öğrencinin sonradan vurguladığı gibi burada **sıkıştırılan metin değil, metni sınıflandıran modeldir**.

3.2 Product quantization

Belgeler ya da cümleler uzayda yüksek boyutlu vektörlerle temsil edilir. Sınıflandırma için bu vektörler arasındaki mesafeler önemlidir: örneğin spor haberleri birbirine, siyaset haberleri birbirine, magazin haberleri birbirine yaklaşarak kendi aralarında gruplaşır. Nicemlemede (quantization) uzay bölgelere ayrılır ve her bölge, vektörlerin yakınsadığı bir merkezle, yani bir **kod kitabı** (codebook) elemanıyla temsil edilir. Slaytta noktaların çokgen bölgelere ayrıldığı ve her bölgenin merkezinin bir çarpıyla işaretlendiği bir çizim gösterildi (ekrandan, 00:25:20).

Hoca codebook için "sözlük" karşılığını önerdi. Sunan öğrenci, bunun anlamlı kelimelerden oluşan bir sözlük olmadığını, kod kitabında "ABD" ya da "ABL" gibi stringlerin yer alabileceğini belirtti; hoca da kelimenin kullanıldığı yere göre farklı anlam alabileceğini ekledi.

> **Ek bilgi (derste anlatılmadı):** Product quantization'da büyük olasılıkla vektör alt parçalara bölünür ve her alt parça ayrı bir kod kitabıyla nicemlenir; vektör yalnız kod kitabı indeksleriyle saklandığı için çok az yer kaplar.

3.3 Retraining

İkinci yöntem yeniden eğitimdir: orijinal model nicemlenir, nicemlenmiş modelle bir kez daha eğitim yapılır, ardından model yeniden nicemlenir. Bu sayede biraz daha sıkıştırma ve doğruluk kazanılır.

3.4 Metne özel yöntem: pruning

Önceki iki yöntem metne özel değildi. Pruning (budama) ise metin sınıflandırma modeline özgüdür. Modelin bir kelime hazinesi (vocabulary) vardır ve bunun büyük kısmı bilgi değeri düşük kelimelerden oluşur. Sorun bir özellik seçimi (feature selection) problemine dönüşür: sözlüğü küçültmek için bilgi değeri yüksek kelimelerin ayrılması gerekir. Bunun için K adet kelime ya da n-gram (yan yana gelen n kelime) seçilir. Seçim kelime vektörlerinin **norm** değerine göre yapılır; norm, vektörün büyüklüğüdür ve sunan öğrenciye göre vektörün taşıdığı bilgiyle doğru orantılıdır (ekrandan, 00:30:00).

Slayttaki tablo iki yarıdan oluşur: sol yarı entropi sıralamasına göre ilk on kelimeyi, sağ yarı norm sıralamasına göre ilk on kelimeyi gösterir (ekrandan, 00:30:00):

Entropi sırası	Kelime	Norm sırası	Norm sırası	Kelime	Entropi sırası
1	.	354	1	mediocre	1399
2	,	176	2	disappointing	454
3	the	179	3	so-so	2809
4	and	1639	4	lacks	1244
5	i	2374	5	worthless	1757
6	a	970	6	dreadful	4358
7	to	1775	7	drm	6395
8	it	1956	8	poorly	716
9	of	2815	9	uninspired	4245
10	this	3275	10	worst	402

Örneğin "disappointing" kelimesinin norm sırası 2'dir; ilk K kelime seçildiğinde bu kelime seçilenler arasına girer. Kelimeler büyük olasılıkla bir film yorumu veri setinden gelmektedir ve olumsuz yorumu ayırt etmede yüksek katsayıya sahiptir.

Hoca tabloyu yorumlarken, entropi sütununda the, and, i gibi sık geçen kelimelerin daha düşük entropiyle daha az bitle kodlanacağını, mediocre ya da disappointing gibi nadir kelimelerin ise yüksek entropi değerleri taşıdığını söyledi. Sunan öğrenci burada amacın sık tekrar edenleri sıkıştırmak olmadığını, sınıflandırmada daha önemli olan norm değeri yüksek kelimeleri koruyarak sözlüğü küçültmek olduğunu açıkladı. Ardından hoca, tablodaki sayıları entropi değeri sandığını, oysa iki sütunun da değer değil **sıralama** (ranking) olduğunu fark ettiğini belirtti.

Pruning'de bir sorun vardır: seçilen K kelimenin hiçbiri bir belgede bulunmayabilir. Örneğin seçilen altı kelime (dreadful, worthless, lacks, disappointing, mediocre gibi) bir belgede hiç geçmeyebilir. Çözüm olarak o belgedeki en yüksek norm değerine sahip kelime sözlüğe eklenir (ekrandan, 00:30:00).

Son olarak oluşturulan sözlüğe hashing uygulanır; sunan öğrenciye göre bu 1-2 MB'lık ek bir kazanç sağlar.

3.5 Deneyler

Sunulan grafiklerde mavi noktalar FastText.zip'i, kırmızılar diğer yöntemleri gösterir. Grafiklerde sol üst köşe idealdir: küçük boyut ve yüksek doğruluk. Pruning miktarı arttıkça boyut düşer ve sola gidilir, ancak doğruluk kaybı da artar. Bazı veri setlerinde (örneğin DBpedia) sözlük küçültülse de kayıp çok az olurken, Amazon Polarity veri setinde pruning arttıkça doğruluk kaybı belirginleşti. Altyazıya göre bir tabloda doğruluk 92,1'den 91,4'e, bir başkasında 89,1 civarına düştü; sunan öğrenci bunu mevcut performansın büyük ölçüde korunduğu biçiminde yorumladı.

Daha büyük ve zor bir veri setinde yalnız nicemlemenin, normun ve yeniden eğitimin etkisi gösterildi; bu tabloda hashing ve pruning yoktur (ekrandan, 00:35:50):

Model	k	norm	retrain	Doğruluk	Boyut
full (sıkıştırılmamış)				45,4	12 GiB
Input	128			45,0	1,7 GiB
Input	128	x		45,3	1,8 GiB
Input	128	x	x	45,5	1,8 GiB
Input+Output	128	x		45,2	1,5 GiB
Input+Output	128	x	x	45,4	1,5 GiB

Yalnız nicemlemeyle model 12 GiB'tan 1,7 GiB'a iner ve doğruluk 45,4'ten 45,0'a düşer. Yeniden eğitimle doğruluk 45,5'e çıkar.

Soru (sunan öğrenci): Tablodaki "Output" ne anlama geliyor?

Cevap: Hoca çıktıyı açılmış (decompress edilmiş) veri olarak anladığını söyledi. Sunan öğrenci, makalede çıktıda oluşan matrisin de nicemlendiğine dair bir ifade olduğunu, ancak nedenini çözemediğini, 300 MB'lık kazancın buradan gelebileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Doğrusal bir sınıflandırıcı olan FastText'te büyük olasılıkla iki matris vardır: kelime gömmelerini tutan girdi matrisi ve gömmeyi sınıf puanlarına dönüştüren çıktı (sınıflandırıcı) matrisi. "Input+Output" satırları iki matrisin birlikte nicemlendiğini gösterir; sınıf sayısı çok olduğunda çıktı matrisi de büyüdüğü için tablodaki 1,8 GiB'tan 1,5 GiB'a inişin kaynağı büyük olasılıkla budur.

3.6 Gelecek çalışmalar ve sonuç

Makaleye göre entropi sıralaması ile norm sıralaması birleştirilebilir: entropi sıralaması yüksek ama normu düşük kelimeler birlikte değerlendirildiğinde farklı bir sıralama çıkabilir. Entropi ile pooling'in birlikte kullanılması da önerilmiştir. Yazarlar bunların her birinin ayrı bir çalışma konusu olduğunu belirtmiştir.

Sonuç olarak pruning, hashing ve nicemleme ile FastText modeli sıkıştırılmış, bunu yaparken doğruluk ve hızda kayda değer bir değişiklik olmamıştır.

Soru: Makale entropinin nasıl hesaplandığına dair bir formül veriyor mu?

Cevap: Hayır. Makalede nicemleme ve pruning'de K kelimenin nasıl seçildiğine dair formüller vardır, entropi için yalnız bir atıf bulunur. Pruning norm üzerinden, yani kelimenin taşıdığı bilgi üzerinden yapılır.

4. Sunum 3: Fraktal Görüntü Sıkıştırmaya Giriş (İngilizce sunum)

Üçüncü sunumda bir öğrenci fraktal görüntü sıkıştırmaya giriş niteliğinde bir makaleyi İngilizce anlattı. Altyazı bu bölümde büyük ölçüde anlamsız olduğu için yalnız slaytlarda okunan ve altyazıda seçilebilen içerik alındı. Aynı makale bir sonraki sunumda Türkçe anlatıldı.

Seçilebilen ana fikirler şunlardır: fraktal, farklı ölçeklerde kendine benzeyen karmaşık örüntülerdir ve basit bir sürecin sürekli bir geri besleme döngüsünde tekrarlanmasıyla oluşur. Fraktal görüntü sıkıştırma, görüntü içindeki kendine benzerlikten (self-similarity) yararlanır: kodlanacak görüntü için $W_1, W_2, \ldots$ dönüşümlerinden oluşan bir koleksiyon aranır; benzer bloklardan yalnız biri ve onu diğerine dönüştüren dönüşüm saklanır.

Slayttaki kodlama düzeni (ekrandan, 00:45:10):

- 256 gri seviyeli 128×128 bir görüntü **range image** olarak adlandırılır.
- Orijinal görüntüde ortalama alınarak (aşağı örnekleme ve alçak geçiren süzme) 64×64 boyutuna indirilir; bu yeni görüntü **domain image** olur.
- Her bloğa bir afin dönüşüm uygulanır; amaç, verilen range bloğuna en iyi yaklaşan domain bloğu dönüşümünü bulmaktır.
- Domain görüntüsünde 4×4 bloklar (i, j), $i, j \in [1, 16]$ ile; range görüntüsünde 4×4 bloklar (k, l) ile indekslenir.

Sonuçlar slaytına göre (ekrandan, 00:50:00) 128×128 Lena görüntüsü 2×2 ve 4×4 bloklarla çözülmüştür. Küçük range bloklarla (8'den küçük) kodlandığında SNR yüksek, sıkıştırma oranı düşüktür. Daha büyük bloklarla sıkıştırma artar ama görüntüdeki ayrıntılar kaybolur.

Sunumun sonunda hoca, sunan öğrenciye makaleyi ne kadar anladığını ve ana fikrin ne olduğunu sordu. Cevaba göre ana fikir, görüntüdeki birbirine benzeyen kısımları bulup dönüşümlerle ifade ederek boyutu azaltmaktır; matematiksel kısım zordur.

5. Sunum 4: Fraktal Görüntü Sıkıştırma (aynı makalenin Türkçe sunumu)

Dördüncü sunumda başka bir öğrenci fraktal görüntü sıkıştırmayı anlattı. Sunum sonunda iki sunumun aynı makaleye dayandığı anlaşıldı; hoca makale başlıklarının farklı yazılmış olması nedeniyle bunun gözden kaçtığını belirtti. Makale fraktallara dayalı görüntü kodlamayı anlatır ve basit bir algoritma geliştirir; sunan öğrenciye göre makaledeki kodlama sistemi Jacquin'in önerdiği fikirlerden geliştirilmiştir (altyazıda "Arnold Jack Quinn"; büyük olasılıkla Arnaud Jacquin).

5.1 Giriş

Teknolojinin ilerlemesiyle büyük bilgi depolama ve hızlı iletişim ihtiyacı artmıştır. Görüntülerin daha az bellekte saklanması depolama maliyetini doğrudan azaltır ve veri iletimini hızlandırır. Görüntü, pikselleri temsil eden bit koleksiyonlarıdır. Orta kalitede görüntüler için bile çok sayıda piksel ve bit gerekir; sunan öğrenci bu bitleri görüntüyü belirleyen sorulara verilen "evet" ya da "hayır" yanıtlarına benzetti.

5.2 Fotokopi makinesi benzetmesi

Fraktal sıkıştırmayı anlamak için özel bir fotokopi makinesi düşünülür: makine kopyalanacak görüntünün boyutunu yarıya indirir ve onu üç kez çoğaltır. Bu makinenin çıktısı tekrar tekrar girdi olarak verilirse ne olur? Makine her seferinde görüntüyü küçülttüğü için, makineyi tekrar tekrar çalıştırmak ilk görüntüyü belli bir noktaya indirger. Aslında değişen yalnız görüntünün konumu ve yönüdür. Bir yüz ilk çıktıda üç yüze, sonra dokuza, sonra yirmi yediye dönüşür; sayı üstel artar, her seferinde görüntü küçülür.

Girdinin nasıl dönüştürüldüğü, makine geri besleme döngüsünde çalıştırıldığında nihai sonucu belirler. Dönüşümler üzerinde tek bir kısıt vardır: dönüşüm görüntüyü küçültmelidir. Aksi halde kopya sayısı sınırsız olduğundan nihai görüntü sonsuz boyuta ulaşırdı.

5.3 Afin dönüşümler

Uygulamada dönüşümler düzlemin **afin dönüşümleri** arasından seçilir (ekrandan, 01:24:40):

$$w_i \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} a_i & b_i \\ c_i & d_i \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} e_i \\ f_i \end{bmatrix}$$

Bu dönüşümler giriş görüntüsünü eğebilir, uzatabilir, döndürebilir, ölçekleyebilir ve öteleyebilir. Geri besleme döngüsünde çalışan bu dönüşümlerin ortak özelliği, oluşan görüntünün kendisinin dönüştürülmüş ve küçültülmüş kopyalarından oluşması ve dolayısıyla **her ölçekte ayrıntıya sahip olmasıdır**; yani görüntüler fraktaldır. Sunan öğrenciye göre bu fraktal üretme yöntemi Hutchinson'a aittir.

5.4 Dönüşüm koleksiyonu olarak saklamanın kazancı

Barnsley (altyazıda "Bensley"), görüntüleri dönüşüm koleksiyonları olarak saklamanın görüntü sıkıştırmaya yol açabileceğini öne sürmüştür. Karmaşık görünen bir görüntü yalnız birkaç afin dönüşümle üretilebilir. Her dönüşüm 6 sayıyla $(a_i, b_i, c_i, d_i, e_i, f_i)$ tanımlanır (ekrandan, 01:26:25):

- Verilen: 4 dönüşüm, dönüşüm başına 6 sayı, sayı başına 32 bit.
- İşlem: $4 \times 6 \times 32$.
- Sonuç: 768 bit.

Aynı görüntüyü piksel koleksiyonu olarak saklamak ise slayttaki çözünürlük için en az 65.536 bit ister (ekrandan, 01:26:25).

5.5 Büzücü dönüşümler ve sabit nokta teoremi

Bir dönüşüm, herhangi iki nokta için aralarındaki uzaklığı azaltıyorsa **büzücüdür** (contractive). Hoca slayttaki "sözleşmeli dönüşüm" çevirisinin İngilizce karşılığını sordu; karşılığın contractive olduğu netleşti. Tanım (ekrandan, 01:28:30):

$$d\big(w(P_1), w(P_2)\big) < s\, d(P_1, P_2), \quad s < 1$$

Burada d uzaklıktır. P_1 ve P_2 iki nokta olsun; dönüşümden sonra aralarındaki uzaklık her zaman orijinalden küçüktür, hem de 1'den küçük bir çarpanla küçülür.

Büzülme haritası sabit nokta teoremi sezgisel olarak açık bir şey söyler: noktalar her dönüşümde birbirine biraz daha yaklaşıyorsa, dönüşüm sayısı artırıldıkça noktalar sonunda tek ve benzersiz bir sabit noktaya yakınsar. Bu basit görünümlü teorem, bir görüntüyü bir dönüşümler koleksiyonuyla nasıl tanımlayabileceğimizi anlatır: hangi görüntüyle başlanırsa başlansın, dönüşüm tekrar tekrar uygulanınca aynı sabit görüntüye yaklaşılır.

5.6 Neden "fraktal"?

Şema bir görüntüyü fotokopi makinesi benzetmesine çok benzeyen bir dönüşümler koleksiyonu olarak kodlar. Eğrelti otunun her ölçekte ayrıntıya sahip olması gibi, dönüşümlerden yeniden oluşturulan görüntü de öyledir. Kodu çözülmüş görüntünün doğal bir boyutu yoktur; herhangi bir boyutta çözülebilir. Daha büyük boyutta çözme için gereken ekstra ayrıntı kodlama dönüşümleri tarafından otomatik olarak üretilir (ekrandan, 01:30:00).

Bu ayrıntının gerçek olup olmadığı sorulabilir: bir kişinin görüntüsünü her yinelemede büyüterek sonunda deri hücrelerini ya da atomları görebilir miyiz? Cevap elbette hayırdır. Üretilen ayrıntı, görüntü dijitalleştirildiğinde var olan gerçek ayrıntıyla ilişkili değildir; yalnız büyük ölçekli özellikleri kodlayan dönüşümlerin bir ürünüdür. Bazı durumlarda düşük büyütmelerde bu ayrıntı gerçekçi görünür. Sunumda Lena fotoğrafının fraktal kodlamasından büyütülmüş bir ayrıntı gösterildi: çok fazla ayrıntı görünmüyor, ama belirgin bir kayıp da oluşmuyor.

5.7 Sıkıştırma oranının ölçülmesindeki zorluk

Görüntü herhangi bir ölçekte çözülebildiğinden fraktal şema için sıkıştırma oranını ölçmek zordur. Sunumdaki örnek:

- Verilen: Lena görüntüsü 5,7:1 oranında sıkıştırılmıştır; görüntü orijinal boyutunun 4 katı ölçekte çözülmüştür.
- İşlem: kenarlar 4 kat büyüdüğü için piksel sayısı $4 \times 4 = 16$ kat artar (ara adım eklendi). Görünen oran $5{,}7 \times 16$ olur.
- Sonuç: $91{,}2 : 1$.

Bu bir hile gibi görünür; çünkü büyütülmüş görüntüdeki her ölçekteki ayrıntı gerçek değildir.

Soru (hoca): Fraktal sıkıştırma kayıplı mı, kayıpsız mı?

Cevap: Kayıplıdır; ancak kayıplar iyi tolere edildiği için çok göze batmaz.

5.8 Görüntünün kodlanması

Önceki teoremler, W dönüşümünün bütün görüntülerin uzayında benzersiz bir sabit noktası olacağını söyler: hangi görüntüyle başlanırsa başlansın W tekrar tekrar uygulanınca sabit bir görüntüye yaklaşılır. O halde bir F resmi verildiğinde, W_1'den W_n'e dönüşümlerin birleşimi olan W öyle bulunmalıdır ki F, W'nin sabit noktası olsun. Bunun için F parçalara ayrılır, parçalara W_i dönüşümleri uygulanır ve F geri elde edilir.

Bu, görüntünün kendi içinde benzerlikler taşımasına dayanır. Lena fotoğrafında şapkanın kenarı ile omzun kenarı birbirine benzer; arkadaki aynaya yansıyan görüntü de benzer bölgeler oluşturur. Bu benzer kısımlar sıkıştırma için kullanılır.

Makaledeki örnek kodlama şöyledir:

- Her pikseli 256 gri seviyeden biri olabilen 128×128 bir görüntü **range image** olarak alınır.
- Orijinal görüntünün ortalaması alınarak (aşağı örnekleme ve alçak geçiren süzme) 64×64 boyutunda bir **domain image** elde edilir.
- İki görüntü de 4×4 piksellik bloklara bölünür.

Her dönüştürülmüş domain bloğu şu formülle elde edilir (ekrandan, 01:34:50):

$$\tilde{D}_{i,j} = \alpha D_{i,j} + t_0, \quad \alpha \in [0,1],\ \alpha \in \mathbb{R}, \quad t_0 \in [-255, 255],\ t_0 \in \mathbb{Z}$$

Ardından her range bloğu $R_{k,l}$, bir bozulma ölçüsü kullanılarak bütün dönüştürülmüş domain bloklarıyla karşılaştırılır ve her range bloğuna en yakın domain bloğu bulunur. Sunan öğrenci bozulma ölçüsünün matematiksel ayrıntısını hızlı geçti.

Bulunan en iyi dönüştürülmüş domain bloğu ilgili range bloğuna atanır: domain bloğunun koordinatları, α ve t_0 ile birlikte bir dosyaya yazılır. Bu dosya **fraktal kod kitabıdır**.

5.9 Kod çözme

Kod çözmede orijinal görüntü, fraktal kod kitabında tanımlı dönüşümlerin herhangi bir başlangıç görüntüsüne yinelemeli olarak uygulanmasıyla yeniden oluşturulur. Bir girdi dönüşüme verilir ve birinci çıktı elde edilir; birinci çıktı dönüşüme verilir ve ikinci çıktı elde edilir; bu n. yinelemeye kadar sürer ve görüntü kodlanmış görüntüye yakınsar.

5.10 Sonuçlar

128×128 Lena görüntüsü 2×2 ve 4×4 blok boyutlarıyla kodlanıp çözüldü (ekrandan, 01:37:05):

	Yöntem I	Yöntem II
Blok boyutu	2×2	4×4
Yineleme sayısı	6	6

	Yöntem I	Yöntem II
Kodlama süresi	10 dk	55 s
Kod çözme süresi	45 s	28 s
Kod kitabı boyutu	16.384 bayt	6.144 bayt
SNR	27 dB	21 dB
Tepe hatası	115	95
Bit oranı	1 bayt/piksel	0,375 bayt/piksel

Slayttaki dipnota göre tepe hatası, orijinal ile kodu çözülmüş görüntü arasındaki en büyük piksel farkıdır; deneyler 386/25 MHz işlemcili, 4 MB RAM'li bir bilgisayarda yapılmıştır (ekrandan, 01:37:05).

Bit oranları kod kitabı boyutundan doğrulanabilir (ara adım eklendi): görüntüde $128 \times 128 = 16.384$ piksel vardır. $16.384/16.384 = 1$ bayt/piksel ve $6.144/16.384 = 0{,}375$ bayt/piksel.

> **Ders sonrası not:** Sunan öğrenci 4×4 blok için "piksel başına 0,3 bayt" dedi; slayttaki tabloya göre doğrusu 0,375 bayt/pikseldir.

2×2 blokta kod kitabı daha büyüktür, kodlama ve kod çözme daha uzun sürer, ancak SNR daha yüksektir. 4×4 blok çok daha az yer kaplar ve hızlıdır, ancak SNR daha düşüktür. Sunan öğrenci sıkıştırma açısından 4×4'ün çok daha avantajlı göründüğünü söyledi. Sonuçlar MATLAB ile elde edilmiştir; gerçek donanımla süreler kısaltılabilir.

Önerilen tasarımın zayıflığı, range ve domain görüntüleri için **sabit boyutlu blok** kullanmasıdır. Görüntülerde kodlanması diğerlerinden zor bölgeler vardır, örneğin Lena fotoğrafındaki gözler. Bu yüzden blok boyutunu kodlama hatasına göre ayarlayan bir mekanizma gerekir.

5.11 Avantaj ve uygulama alanları

Fraktal kod çözmenin en büyük avantajlarından biri, kodu çözülmüş görüntü üzerinde yakınlaştırma ve uzaklaştırma yapılırken görüntü kalitesinin çok bozulmamasıdır. Bu özellik, doktorların görüntü ayrıntılarına odaklanması gereken tıbbi uygulamalarda ya da güvenlik kamerası görüntülerinde bir yüzün netleştirilmesi gereken sistemlerde kullanılabilir. Sunan öğrenciye göre bu, JPEG ya da MPEG'de kullanılan ayrık kosinüs dönüşümü tabanlı algoritmalara göre açık bir avantajdır.

6. Sunum 5: Tekrarlayan Sinir Ağlarıyla Tam Çözünürlüklü Görüntü Sıkıştırma

Son sunumda bir öğrenci, Google Research ekibinin "Full Resolution Image Compression with Recurrent Neural Networks" makalesini anlattı. Sunuma göre makale 2017'de CVPR'da (bilgisayarlı görü alanının en önemli konferanslarından biri) yayımlanmıştır.

6.1 Motivasyon

Görüntü sıkıştırma, sinir ağlarının başarılı olacağı düşünülen alanlardan biriydi, ancak uzun süre bunu gösteren çalışma yoktu. 2016'da ICLR konferansında tek bir sinir ağıyla en iyi yöntemlerden (state of the art) daha iyi sonuç veren bir mimari önerildi. Ancak bu mimari 32×32 görüntülerle sınırlıydı. Bu çalışmanın amacı farklı boyutlardaki görüntülerde de en iyi yöntemlerden iyi sonuç veren bir mimari geliştirmektir.

6.2 Otokodlayıcılar ve sabit sıkıştırma oranı sorunu

Sunan öğrenci, makalede olmayan ama anlamak için eklediği bir slaytla otokodlayıcıyı (autoencoder) anlattı: el yazısıyla yazılmış bir "4" görüntüsü kodlayıcıya verilir, kodlayıcı onu sıkıştırır, kod çözücü tekrar açar. Otokodlayıcılar görüntü boyutunu azaltmak, görüntüleri sıkıştırmak ve görsel temsil (visual representation) çıkarmak için kullanılıyordu; yakın zamanda 64×64 görüntüler için de kullanılmıştı.

Sorun şudur: otokodlayıcıda kodlayıcının çıktısı, yani **darboğaz** (bottleneck) kısmı sabit boyutlu olduğu için sıkıştırma oranı da sabittir ve farklı boyutlardaki görüntüleri sıkıştırmak zorlaşır. Bu çalışma, her görüntü için sabit bir sıkıştırma oranı olmamasını ve farklı piksel boyutlarındaki görüntülerin sıkıştırılabilmesini hedefler.

6.3 Yöntem

Ağ üç parçadan oluşur: kodlayıcı E, ikilileştirici (binarizer) B ve kod çözücü D. E ve D tekrarlayan ağ bileşenleri içerir. Giriş görüntüsü önce kodlanır, sonra saklanabilecek ya da iletilebilecek ikili kodlara dönüştürülür. Kod çözücü bu ikili koddan orijinal görüntünün bir tahminini üretir ve işlem tekrar edilir (ekrandan, 01:45:00).

Yinelemeli şema (ekrandan, 01:45:00):

$$b_t = B(E_t(r_{t-1})), \qquad \hat{x}_t = D_t(b_t) + \gamma\,\hat{x}_{t-1}$$

$$r_t = x - \hat{x}_t, \qquad r_0 = x, \qquad \hat{x}_0 = 0$$

Burada D_t ve E_t, t. yinelemedeki kod çözücü ve kodlayıcıdır; b_t t. yinelemedeki ikili gösterimdir; $\hat{x}_t$ orijinal görüntü x'in aşamalı olarak yeniden yapılandırılmasıdır; r_t ise x ile $\hat{x}_t$ arasındaki **artıktır** (residual). İlk yinelemede çıktı sıfır olduğu için artık doğrudan görüntünün kendisidir ($r_0 = x$). $\gamma = 0$ ise yöntem one-shot, $\gamma = 1$ ise additive yeniden yapılandırmadır.

> **Ders sonrası not:** Sunan öğrenci yöntemi bir an "loss-free compression" diye nitelendirdi, hemen ardından da görüntünün orijinale en yakın biçimde geri çevrilmeye çalışıldığını söyledi. Yöntem kayıplıdır: çıktı orijinalin bir yaklaşığıdır ve kalite yineleme sayısıyla, yani harcanan bit sayısıyla artar.

Sunan öğrencinin anlattığı tek bir yinelemenin yapısı şöyledir:

```
Kodlayıcı:   x -> Evrişim -> RNN -> RNN -> RNN -> İkilileştirici (B)
Kod çözücü:  b -> Evrişim -> RNN -> RNN -> RNN -> RNN -> Evrişim -> x̂
```

Bu yapı tek bir yinelemedir; işlem tekrarlandıkça başarım artar.

6.4 RNN türleri

Çalışmada dört tür tekrarlayan birim kullanıldı: LSTM (Long Short-Term Memory), Associative LSTM, GRU (Gated Recurrent Unit) ve Residual GRU. Bunlar kodlayıcıda kodlama, kod çözücüde görüntüyü geri elde etme amacıyla kullanılır. Sunan öğrenci bu birimlerin ortak özelliğini şöyle anlattı: t anındaki durum işlenirken $t-1$ anındaki durum da hesaba katılır. Bu yüzden video ve metin işlemede de kullanılırlar: videoda önceki kare, metinde önceki kelime işin içine katılır; sonuç yalnız tek bir piksel ya da kareden çıkarılmaz.

6.5 Yeniden yapılandırma çerçeveleri

Kod çözücü tarafında görüntüyü yeniden kurmak için üç çerçeve geliştirildi:

- **One-shot ($\gamma = 0$):** her yinelemede orijinal görüntünün tamamı yeniden yapılandırılmaya çalışılır.
- **Additive ($\gamma = 1$):** her yinelemede yalnız önceki yinelemeden kalan artık görüntü yeniden yapılandırılır; nihai görüntü bütün yinelemelerin çıktılarının toplamıdır.
- **Residual scaling:** önceki iki yöntemde artık başlangıçta çok büyüktür ve yinelemelerle azalması beklenir. Ancak kodlayıcı ve kod çözücünün geniş bir değer aralığında verimli çalışması zordur; yüksek bir hatadan başlandığı için hatayı düzeltmek de zorlaşır. Bu yüzden önceki yinelemenin yeniden yapılandırılmış çıktısından her 32×32 yama (patch) için bir kazanç çarpanı üretilir ve mevcut yinelemeye giren artık bu kazançla ölçeklenir.

Residual scaling'in denklemleri (ekrandan, 01:50:00):

$$g_t = G(\hat{x}_t), \qquad b_t = B\Big(E_t\big(r_{t-1} \odot \mathrm{ZOH}(g_{t-1})\big)\Big)$$

$$\hat{r}_{t-1} = D_t(b_t) \oslash \text{ZOH}(g_{t-1}), \qquad \hat{x}_t = \hat{x}_{t-1} + \hat{r}_{t-1}, \qquad r_t = x - \hat{x}_t$$

$$g_0 = 1, \qquad r_0 = x$$

$G()$, kazanç faktörü g_t'yi tahmin eden beş katmanlı bir evrişimli ağdır. İlk dört katman ELU doğrusal olmayanlığıyla 3×3 evrişim çekirdeği kullanır ve 32 çıkış derinliği verir. Son katman 2×2 evrişim çekirdeğiyle 1 çıkış derinliği verir. ELU'nun aralığı $(-1, \infty)$ olduğundan, g_t'yi $(1, \infty)$ aralığında elde etmek için ağın çıkışına $+2$ sabiti eklenir (ekrandan, 01:50:00).

> **Ek bilgi (derste anlatılmadı):** ZOH (zero-order hold) büyük olasılıkla her yama için tek bir sayı olan kazancı, o yamanın bütün piksellerine kopyalayarak görüntü boyutuna genişletir. $\odot$ eleman eleman çarpma, $\oslash$ eleman eleman bölmedir; artık kodlanmadan önce büyütülür, çözüldükten sonra aynı çarpanla küçültülür.

Soru (hoca): ELU nedir?

Cevap: Sunan öğrenci slaytta ELU yazdığını, bunun ReLU ile karıştırılmış olabileceğini düşündüğünü söyledi. Hoca ELU'nun Exponential Linear Unit olduğunu belirtti.

6.6 Entropi kodlaması

İkilileştiricinin çıktısı üzerinde entropi kodlaması da yapılabilir. Bunun için PixelRNN mimarisinden esinlenen ikili bir RNN mimarisi geliştirilmiştir. Entropi iki yöntemle tahmin edilir: **tek yinelemeli entropi kodlayıcı** (single iteration entropy coder) her yinelemede entropiyi ayrı hesaplar; **aşamalı entropi kodlayıcı** (progressive entropy coder) önceki yinelemelerin bilgisini de kullanır. Kodlanmış ikili kod üzerinde evrişim gezdirilir, LSTM'e verilir ve entropi tahmin edilir; aşamalı yöntem bunu yinelemeler boyunca üst üste yapar.

6.7 Veri setleri ve eğitim

İki eğitim veri seti kullanıldı:

- Önceki çalışmalarda kullanılan 32×32 görüntülerden oluşan veri seti.
- 1280×720 çözünürlüklü yaklaşık 6 milyon görüntüden kurulan veri seti: görüntüler küçük yamalara bölünür ve PNG ile sıkıştırıldığında en kötü orana sahip 100 yama seçilir. Böylece ağ, PNG'nin bile zorlandığı en zor görüntülerle eğitilir. Sıkıştırılması zor olduğu için bu sete **yüksek entropili veri seti** denir.

Modeller Google'ın kendi ürünü olan TensorFlow ile eğitildi. Adam optimizer kullanıldı ve öğrenme oranı eğitim sırasında değiştirildi: başta hızlı öğrenmesi için büyük bir değerden başlanıp öğrenme ilerledikçe düşürüldü.

RNN'in açılım (unroll) yineleme sayısı 16 alındı. Yaklaşık bir milyon eğitim adımı yapıldı; sunan öğrenci bunun çok fazla olduğunu, makalenin kullanılan donanım ve enerji hakkında bilgi vermediğini belirtti.

6.8 Değerlendirme ölçütleri

İki ölçüt kullanıldı:

- **MS-SSIM** (Multi-Scale Structural Similarity): 0 ile 1 arasında puan verir.
- **PSNR-HVS** (insan görme sistemine göre uyarlanmış tepe sinyal-gürültü oranı): desibel cinsinden ölçülür.

İkisinde de yüksek değer daha iyidir. Modelleri sıralamak için bit oranına göre çizilen kalite eğrisinin altında kalan alan (AUC) hesaplandı. Testler Kodak Photo CD veri setinde (24 görüntü) yapıldı. 4 RNN türü × 3 yeniden yapılandırma çerçevesi = 12 farklı model denendi ve en başarılı sekizi raporlandı.

6.9 Sonuçlar

Kodak veri setinde 2 bit/piksele kadar AUC sonuçları (entropi kodlaması olmadan) (ekrandan, 01:54:10):

32×32 veri setiyle eğitim	MS-SSIM sırası	MS-SSIM AUC	PSNR-HVS sırası	PSNR-HVS AUC
GRU (One Shot)	1	1,8098	1	53,15
LSTM (Residual Scaling)	2	1,8091	4	52,36
LSTM (One Shot)	3	1,8062	3	52,57
LSTM (Additive Reconstruction)	4	1,8041	6	52,22
Residual GRU (One Shot)	5	1,8030	2	52,73
Residual GRU (Residual Scaling)	6	1,7983	8	51,25
Associative LSTM (One Shot)	7	1,7980	5	52,33
GRU (Residual Scaling)	8	1,7948	7	51,37
Baseline		1,7225		48,36

Yüksek entropili veri setiyle eğitim	MS-SSIM sırası	MS-SSIM AUC	PSNR-HVS sırası	PSNR-HVS AUC
LSTM (One Shot)	1	1,8166	8	48,86
GRU (One Shot)	2	1,8139	2	53,07
Residual GRU (One Shot)	3	1,8119	1	53,19
Residual GRU (Residual Scaling)	4	1,8076	7	49,61
LSTM (Residual Scaling)	5	1,8000	4	51,25
LSTM (Additive)	6	1,7953	5	50,67
Associative LSTM (One Shot)	7	1,7912	3	52,09
GRU (Residual Scaling)	8	1,8065	6	49,97
Baseline LSTM		1,7408		48,88

JPEG	MS-SSIM AUC	PSNR-HVS AUC
YCbCr 4:4:4	1,7748	51,28
YCbCr 4:2:0	1,7998	52,61

32×32 veri setinde GRU (One Shot) iki ölçütte de birinci oldu. Yüksek entropili veri setinde GRU (One Shot) iki ölçütte de ikinci, Residual GRU (One Shot) PSNR-HVS'de birinci oldu. Sunan öğrencinin dikkatini çeken nokta, LSTM (One Shot) modelinin yüksek entropili veri setinde MS-SSIM'e göre birinci, PSNR-HVS'ye göre sonuncu olmasıydı. Öğrenciye göre bu

kadar büyük bir fark, yöntemin genel olarak başarılı olmadığını, yalnız belirli bir ölçüte uygun düştüğünü düşündürür.

Hoca, birinde birinci diğerinde sekizinci olsa da AUC değerleri arasındaki farkın çok büyük olmadığını söyledi. Sunan öğrenci ise bu modelin PSNR-HVS'de baseline LSTM'in de altında kaldığını belirtti; makale de ölçütler çok farklı olduğu için farklı sonuçlar çıkabileceğini yazmıştır.

> **Ders sonrası not:** Hoca farkın yaklaşık yüzde beş olduğunu söyledi. Tabloya göre MS-SSIM AUC farkları gerçekten çok küçüktür (en iyi ile en kötü arasında yaklaşık 0,01). PSNR-HVS'de ise LSTM (One Shot) 48,86, en iyi model 53,19 almıştır; fark yaklaşık 4,33 dB, yani en iyi değerin yaklaşık %8'idir. LSTM (One Shot)'ın 48,86 değeri baseline LSTM'in 48,88 değerinin çok az altındadır.

Entropi kodlaması yapılan ve yapılmayan modellerin karşılaştırmasında, entropi kodlaması yapılmış modeller açıkça daha iyi sonuç verdi. Seçilen dört model de iki ölçütte JPEG'den daha başarılı çıktı.

6.10 Görsel karşılaştırma ve tartışma

Son slaytta JPEG (4:2:0) ile önerilen yöntemin sıkıştırdığı görüntüler yan yana gösterildi: üst satır 0,25 bit/piksel, alt satır 1 bit/piksel. Soldaki görüntüler de orijinal değil, JPEG ile sıkıştırılmış görüntülerdir; orijinal gösterilmemiştir. Yuvarlak içine alınan bölgelerde makale JPEG'de bozulma olduğunu, kendi yönteminde olmadığını göstermektedir.

Hoca, üst satırdaki kırmızı kasklı adam görüntüsünde önerilen yöntemin çıktısında da bozulmayı gözle gördüğünü, arka plandaki yeşil zeminin bozuk olduğunu, hatta soldaki görüntüyü daha net bulduğunu söyledi. Sunan öğrenci, makalenin alt satır için karıncalanmanın daha az olduğunu iddia ettiğini, ayrıca insan gözüne bu konuda güvenilemeyeceğini, çünkü herkesin farklı gördüğünü yazdığını belirtti. Hoca bunu kabul etti: yayınlarda böyle sonuçlar olur; bakan kişi farklı görebilir, ama yazarlar yöntemlerinin daha iyi olduğunu iddia eder.

Örnek Sorular

1. (ekrandan, 00:14:45 ve derste sözlü, 00:20:01) Markov-k kaynağı $X(n) = X(n-1) + X(n-k) \bmod M$ ile üretilmektedir. k büyüdükçe bu veri neden "zor" hale gelir ve DeepZip gibi bir RNN tabanlı sıkıştırıcıyı nasıl etkiler?

> **Cevap (derste verildi):** k büyüdükçe her eleman daha uzaktaki bir elemana bağlı olur; örüntü uzun vadeli hale gelir. Zor veri, ilişkilerin ve örüntülerin tespit edilmesi zor olan veridir. Model bu bağımlılığı öğrenemezse sonraki sembol için yüksek olasılıklı bir tahmin veremez ve sıkıştırma yapamaz. Klasik RNN kaybolan gradyan nedeniyle bu sınıra erken (yaklaşık Markov-35), GRU daha geç, DeepZip-Feat ise ancak Markov-140'ta ulaşır.

2. (derste sözlü, 00:20:57) DeepZip'te aritmetik kodlayıcının kullandığı olasılıklar nereden gelir? Önceden hazırlanmış bir olasılık tablosu var mıdır?

> **Cevap (derste verildi):** Yoktur. Olasılıklar metin ilerledikçe RNN modelinin içinde değişir; RNN önceki bütün sembollerin gizli bilgisinden sonraki sembol için bir dağılım üretir. İlk sembolde ise varsayılan bir dağılım kullanılır.

3. (ekrandan, 00:30:00 ve derste sözlü, 00:30:00) FastText.zip'in pruning tablosunda entropi sıralamasında ilk sıralarda ".", "the", "and" gibi kelimeler, norm sıralamasında ise "mediocre", "disappointing" gibi kelimeler yer almaktadır. Sık geçen kelimeler entropi açısından nasıl kodlanır ve FastText.zip sözlüğü küçültürken neden norma bakar?

> **Cevap (derste verildi):** Sık geçen kelimelerin entropisi düşüktür ve daha az bitle kodlanır; nadir kelimeler daha fazla bit ister. FastText.zip'in amacı ise metni değil sınıflandırma modelini sıkıştırmaktır; bu yüzden sınıflandırmada daha ayırt edici olan, norm değeri yüksek kelimeler korunur ve sözlük bunlarla küçültülür.

4. (ekrandan, 00:35:50) FastText'in sıkıştırılmamış modeli 12 GiB boyutta ve %45,4 doğruluktadır. Girdi matrisi nicemlenmiş model 1,7 GiB boyutta ve %45,0 doğruluktadır. Boyut kaç kat küçülmüş ve doğruluk ne kadar düşmüştür?

Çözüm (derste verilmedi): $12/1{,}7 \approx 7{,}06$ kat küçülme; boyutta yaklaşık %86 azalma. Doğruluk $45{,}4 - 45{,}0 = 0{,}4$ puan düşmüştür. Norm ve yeniden eğitim eklendiğinde doğruluk 45,5'e çıkar ve sıkıştırılmamış modeli de geçer.

5. (ekrandan, 01:26:25) Bir görüntü 4 afin dönüşümle tanımlanmaktadır. Her dönüşüm 6 sayıyla, her sayı 32 bitle saklanmaktadır. Görüntüyü saklamak için kaç bit gerekir?

Cevap (ekrandan verildi): $4 \times 6 \times 32 = 768$ bit. Aynı görüntüyü piksel koleksiyonu olarak saklamak ise en az 65.536 bit ister.

6. (derste sözlü, 01:31:17) Lena görüntüsü fraktal yöntemle 5,7:1 oranında sıkıştırılmış ve orijinal boyutunun 4 katı ölçekte çözülmüştür. Bu durumda görünen sıkıştırma oranı nedir ve bu oran neden yanıltıcıdır?

Cevap (derste verildi): 4 kat ölçekte çözülen görüntü 16 kat daha fazla piksel içerir; görünen oran $5{,}7 \times 16 = 91{,}2 : 1$ olur. Bu bir hile gibidir, çünkü büyütülmüş görüntüdeki ayrıntı gerçek değildir, kodlama dönüşümlerinin ürünüdür.

7. (derste sözlü, 01:31:56) Fraktal görüntü sıkıştırma kayıplı mı, kayıpsız mı bir yöntemdir?

Cevap (derste verildi): Kayıplıdır; ancak kayıplar iyi tolere edildiği için göze çok batmaz.

8. (ekrandan, 01:37:05) 128×128 piksellik bir görüntü fraktal yöntemle kodlanmıştır. 2×2 blokta kod kitabı 16.384 bayt, 4×4 blokta 6.144 bayttır. Her yöntemin bit oranını bayt/piksel cinsinden hesaplayın. Hangisi daha yüksek SNR verir?

Çözüm (derste verilmedi): Görüntüde $128 \times 128 = 16.384$ piksel vardır. 2×2: $16.384/16.384 = 1$ bayt/piksel. 4×4: $6.144/16.384 = 0{,}375$ bayt/piksel. Slayttaki tabloya göre küçük blok (2×2) 27 dB ile daha yüksek SNR verir; 4×4 blok 21 dB verir ama daha az yer kaplar.

9. (ekrandan, 01:50:00) Residual scaling çerçevesinde kazanç ağı $G()$'nin son katmanı ELU kullanır. ELU'nun çıkış aralığı $(-1, \infty)$ ise kazancın $(1, \infty)$ aralığında olması için ne yapılır?

Cevap (ekrandan verildi): Ağın çıkışına $+2$ sabiti eklenir: $(-1 + 2, \infty) = (1, \infty)$.

Dersin Toparlaması

Bu derste beş seminer sunumu yapıldı. İlk sunum, RNN ile aritmetik kodlamayı birleştirerek kayıpsız sıkıştırma yapan DeepZip'ti: RNN önceki sembollerden sonraki sembolün dağılımını tahmin eder, aritmetik kodlayıcı bu dağılımla kodlar; derste görülen PPM ve k-gram modellerinin k sınırı burada aşılır. İkinci sunum, bir metin sınıflandırma modelini product quantization, yeniden eğitim, norm tabanlı budama ve hashing ile küçülten FastText.zip'ti; burada sıkıştırılan metin değil modeldi. Üçüncü ve dördüncü sunum aynı makaleden fraktal görüntü sıkıştırmayı anlattı: afin ve büzücü dönüşümler, sabit nokta teoremi, range ve domain blokları, çözünürlükten bağımsız kod çözme ve blok boyutu ile SNR ve sıkıştırma oranı arasındaki denge. Son sunum, artığı yinelemeli olarak kodlayan RNN tabanlı bir görüntü sıkıştırma ağını ve sonuçlarının MS-SSIM ve PSNR-HVS ile değerlendirilmesini anlattı.

Hoca sunumları derste görülen kavramlara bağladı: entropi ile kelime sıklığı arasındaki ilişki, kod kitabının bir sözlük olarak düşünülmesi, fraktal sıkıştırmanın kayıplı olması ve görsel kalite değerlendirmesinde insan gözünün öznelliği. Seminer sunumları sonraki derslerde de devam edecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.